Part 2 — Challenge 1: put your GEMM inside a language model

In Part 1 you built a fast GEMM for four shapes. Now the reveal: **those four shapes are the matrix multiplies inside a real language model** — a 15-million-number network that writes children's stories. In this challenge you drop your GEMM into that model, run it, and measure the thing that actually matters to a user: **tokens per second.**

This part is the detailed story. If you want the full from-scratch explanation of how a transformer computes, read transformer-from-scratch.md first — it assumes no machine-learning background and builds the whole picture up from "text is a list of integers." The short version is below.

2.1 The model in one screen

Here is a story the model wrote:

> *Once upon a time, there was a little girl named Lily. She loved to play outside in the sunshine. One day, she saw a big, red ball in the sky. It was the sun!* ...
>
> `achieved tok/s: …`

It writes **one token (≈ one word-piece) at a time**: look at all the text so far, score every possible next token, pick one, append, repeat. Each step runs the whole network forward once, and that forward pass is — almost entirely — a sequence of matrix–vector products `y = W·x`, where `x` is the current 288-number hidden state and `W` is a learned matrix. Per token:

Where	Matrix	Shape (`out ← in`)	Count
attention	`Wq, Wk, Wv, Wo`	288 ← 288	4 / layer
FFN	`W1, W3`	768 ← 288	2 / layer
FFN	`W2`	288 ← 768	1 / layer
classifier	`Wcls`	32000 ← 288	1 total

Six layers × 7 + 1 classifier = **43 matrix–vector products per token** — the exact shapes from Part 1. Because the model runs one token at a time, each is a GEMV: **your `student_gemm` with `N = 1`.**

2.2 Meet the storyteller — and measure where its time goes

Get the model and run it (no internet on your node? see the script header for the offline path):

```
cd mini1-gemm
./scripts/setup.sh          # clones the engine, fetches the 60 MB checkpoint, prints a s
```

Now the important part — **don't guess where the time goes, measure it.** `profile.sh` times every matrix multiply, by shape, over a 256-token run:

```
./scripts/profile.sh        # prints a per-shape matmul profile
```

Read *your* profile and answer these in your report — the numbers drive every decision that follows:

1. **What fraction of total runtime is matrix multiply?** Everything else — the norms, the attention softmax, the sampling loop — is the remainder. What does that fraction imply about the *best* speedup you could ever get by accelerating only the GEMMs? (Think Amdahl's law.)

2. **Which shape dominates?** The four shapes map to the classifier (`288→32000`), the FFN (`w1/w3` at `288→768` , `w2` at `768→288`), and the attention projections (`288→288`). The usual saying is "the FFN dominates transformers" — does *your* measurement agree, for a model this small with a 32000-token vocabulary? **Profile, don't assume.**

Carry those two findings — the matmul fraction and the shape ranking — into §2.5; they decide what is worth accelerating and how much it can possibly buy you.

2.3 The seam — plug your GEMM in with one link choice

The model's matrix-multiply routine has this signature (it computes `xout = W·x`):

```
void matmul(float* xout, float* x, float* w, int n, int d);   // xout[d] = W[d×n]·x[n]
```

That is exactly your `student_gemm` with `A = W` (M=d, K=n), `B = x` (N=1), `C = xout` . `src/slm_backend.c` is the entire seam:

```
void matmul(float *xout, float *x, float *w, int n, int d) {
    student_gemm(w, x, xout, /*M=*/d, /*K=*/n, /*N=*/1);
}
```

`scripts/build_slm.sh` compiles the model with its built-in `matmul` routed to your `student_gemm` , and checks that the story is unchanged:

```
./scripts/build_slm.sh                                  # reference GEMM → "IDENTICAL"
GEMM_SRC=path/to/your_gemm.c ./scripts/build_slm.sh     # YOUR GEMM
```

With a correct GEMM the output is byte-for-byte the model's own story — only the speed differs. **A correct kernel is now a working language model.**

2.4 Run it on the accelerator

`build_slm.sh` proved the plumbing on the CPU. To measure tokens/sec on the real accelerator, implement `student_gemm` as an offload to your hardware kernel — `src/student_gemm_fpga.cpp` is a template with the same contract. Two rules decide whether tokens/sec actually goes up:

1. **Load each weight matrix onto the device once** (the template caches it by pointer). The weights never change during generation; re-sending the 37 MB classifier every token would drown everything else.
2. **Only the activation and result cross the bus per call** — `K×N` in, `M×N` out, tens of KB — plus the runtime `M, K, N`.

You build the bitstream; you do not need the U50. Compiling the `.xclbin` is a pure Slurm job on the RCE cluster (≈ 1 h place & route, no FPGA needed to *build*). Only *running* it needs the U50 on the lab node — which you never log in to. You submit the built bitstream, the course runs it on the card, and the **generated story + achieved tok/s** come back to you. `student_gemm_fpga.cpp` is the host seam you fill in (offload `student_gemm` to your kernel); the two rules above — weights resident, only activations crossing the bus — are the point, and that tok/s is your real result.

> **How to build and submit** — the exact commands, the submission portal, and your personal access token are in the **course hardware handout on Moodle** (included in your starter kit). They are class-specific, so they live there rather than on this public page.

2.5 The challenge: make it write faster (tokens/sec)

Run your plugged-in model and read the `achieved tok/s`. Here is where the systems lesson lives, and it is not the one you expect.

First result — it may get *slower*. The model calls `matmul` 43 times per token, each a GEMV (`N = 1`). Two forces work against you:

- **Amdahl's law.** Matrix multiply is the large majority of runtime (you measured the exact fraction *f* in §2.2), so even an *infinitely* fast GEMM caps the speedup at `1 / (1`

- `f`) — and the non-matmul remainder (norms, softmax, the sampling loop) never goes away.
- `N = 1` **is memory-bound.** A GEMV reads the whole weight matrix to make one output vector — no reuse to exploit. This is exactly the case Part 1 warned you is hard.

The fix — raise `N`. Hand the kernel **many** vectors against the same weights per call, turning each GEMV back into a GEMM with reuse. Two honest sources of extra vectors:

- **Prefill** — a prompt of `L` tokens is all known up front, so push all `L` through each matrix at once (`N = L`).
- **Batching** — generate `B` independent stories simultaneously; each step has `B` current vectors (`N = B`), producing `B` tokens per step.

Extend the generation loop to carry `N > 1` vectors and re-measure. **Plot tokens/sec vs `N`** — the curve rising as `N` grows (your Part 1 `N > 1` optimizations finally paying off, launch/overhead amortized across `N` tokens) is the headline result of this challenge. It will flatten eventually — when you're bounded by how fast the weights can be read, no matter how many vectors share them — and naming that ceiling is part of the write-up.

> **The lesson.** A faster kernel is necessary but not sufficient. *How you feed it* — the value of `N` — decides whether your fast GEMM makes the model faster. That is why real inference systems batch, and why Part 1 made you care about the `N > 1` case.

2.6 What to report

1. Your model generating a story through **your** `student_gemm` (build_slm shows `IDENTICAL`), and running on the accelerator — the story + tok/s the portal reports back from your submitted bitstream.
2. **tokens/sec vs `N`** — the single-stream number (possibly below baseline, with your explanation) and the curve as batching/prefill raise `N` .

3. Which matmuls you accelerated (you can offload just the top of the profile — the few biggest shapes you identified in §2.2 are the large majority of matmul time) and the tok/s step from each.
4. Your throughput ceiling and what it would take to raise it — which leads straight into Part 3.

Checkpoint

- [ ] `build_slm.sh` → **IDENTICAL** with your GEMM.
- [ ] Single-stream tok/s, explained honestly (Amdahl + `N=1`).
- [ ] The **tok/s vs N** curve crossing the CPU baseline, with its flat ceiling.